Network Security EE I7000

Final Project Report

Project Title: RSA (Rivest-Shamir-Adleman)

Project Number: 13

Members:

Tanzila Tabassum

Bryan Mgbeojirikwe

Loksharan Saravanan

Table of Contents

Final Project Report 1

Project Title: RSA (Rivest-Shamir-Adleman) *1*

Project Number: 13 1

Members: 1

Tanzila Tabassum 1

Bryan Mgbeojirikwe 1

Loksharan Saravanan 1

1. Introduction 3

2. Objectives 3

3. System Overview 4

4. Technical Implementation Details 4

- **4.1 Mathematical Foundations** 4
- **4.2 Function Breakdown** 4
- **4.3 Input/Output System** 9
- **4.4 Input/Output System Screenshot** 11
- **4.5 Important Technical Details** 13

5. Professor Feedback & Project Adaptation 14

6. Quantum Computing Threat to RSA 14

- **6.1 Vulnerability of RSA** 14
- **6.2 Need for Post-Quantum Cryptography** 15

7. Post-Quantum Cryptographic Techniques 15

- **7.1 Hash-Based Cryptography** 15
- **7.2 Lattice-Based Cryptography** 16

8. Integration with RSA System 16

- **8.1 Hybrid Encryption Strategy** 16
- **8.2 Transition Strategy** 17

9. Challenges in Quantum Migration 18

10. Future Work and Applications 18

11. Conclusion 19

12. References 19

1. Introduction

In today's digital era, data privacy and secure communication have become paramount. Cryptography, the science of secure information exchange, plays a central role in safeguarding sensitive data. Among the most foundational algorithms in public-key cryptography is RSA (Rivest–Shamir–Adleman), which secures millions of internet-based transactions and communications.

This project aims to implement a simplified yet fully functional RSA encryption and decryption system in Python, enabling users to exchange secure messages. Beyond the core implementation, we extend the project to address the emerging threat posed by quantum computing—a technology that could render traditional cryptographic systems obsolete. In light of this, we explore post-quantum cryptographic (PQC) techniques and present strategies for transitioning to quantum-resistant encryption models.

The project not only deepens our understanding of RSA and its mathematical foundations but also equips us with awareness of the evolving landscape of cybersecurity.

2. Objectives

The primary objectives of our project are as follows:

- To design and implement an RSA-based encryption system capable of secure text communication.
- To reinforce theoretical knowledge through hands-on coding, including modular arithmetic, prime validation, and key generation.
- To evaluate the limitations of RSA in the context of emerging computational capabilities, particularly quantum computing.
- To investigate and propose post-quantum cryptographic methods suitable for secure communication in the near future.
- To respond to real-world application concerns, such as integrating RSA/PQC into practical software systems (e.g., encrypted mail clients).

3. System Overview

Our RSA Secure Mail System is a command-line application that offers users the ability to:

- Generate secure RSA key pairs using two user-provided prime numbers.
- Encrypt and save a text message using the RSA algorithm.
- Decrypt a message using a stored private key.
- Persist encryption keys and messages to files for reuse.
  The user-friendly menu-based interface allows for sequential operations including key generation, encryption, and decryption, all backed by foundational cryptographic logic.

4. Technical Implementation Details

4.1 Mathematical Foundations

The RSA system is built upon the following principles:

- The difficulty of factoring large integers: Given two large primes p and q, it is computationally hard to find p and q from their product n = pq.
- The use of modular arithmetic to encrypt and decrypt messages using exponentiation: $c \equiv m^e \bmod n$, and $m \equiv c^d \bmod n$.
- The requirement that e and $\varphi(n)$ (Euler's totient function) be coprime, with d serving as the modular inverse of e.

Our code includes efficient and well-documented helper functions to compute the greatest common divisor (GCD), the modular inverse, and to check for primality using trial division.

4.2 Function Breakdown

- **gcd(a, b):**
  - Implements the Euclidean algorithm to compute the GCD.
  - Returns their greatest common divisor.
  - Repeatedly replaces (a,b) with (b, a mod b) until b becomes 0.

```python
import sys
import math
import random

# ---------------------- Helper Functions ----------------------

def gcd(a, b):
    # Compute the greatest common divisor using Euclidean algorithm
    while b != 0:
        a, b = b, a % b
    return a
```

- **extended_gcd(a, b):**
  - Extended Euclidean algorithm
  - Returns coefficients x and y satisfying ax + by = gcd(a, b), critical for computing modular inverses.

```python
def extended_gcd(a, b):
    # Extended Euclidean algorithm to find modular inverse
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = extended_gcd(b % a, a)
        return (g, x - (b // a) * y, y)
```

- **modinv(a, m):**
  - Finds modular inverse of a modulo m.
  - Returns x such that ax ≡ 1 mod m.
  - Calculating private key d from e and z.

```python
def modinv(a, m):
    # Compute modular inverse of a under modulo m
    g, x, y = extended_gcd(a, m)
    if g != 1:
        return None
    else:
        return x % m
```

- **is_prime(num):**

- Confirms whether a number is prime through trial division up to √num.
- RSA requires two distinct primes for security.

```python
def is_prime(num):
    # Simple primality test
    if num < 2:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True
```

- **generate_keys(p, q):**
  - Computes n = p*q and z = φ(n) = (p-1)(q-1).
  - Finds e (public exponent) coprime with z.
  - Calculates d (private exponent) as modular inverse of e mod z.
  - Prefers common e values first (3,5,17,65537).
  - Enforces minimum n ≥ 128 for ASCII compatibility.
  - Handles cases where no valid e exists.

```python
def generate_keys(p, q):
    # Generate RSA public and private keys from primes p and q
    n = p * q
    z = (p - 1) * (q - 1)
    if n < 128:
        raise ValueError(f"Modulus n={n} is too small - choose larger primes (minimum n ≥ 128)")

    # Choose a small odd integer e such that gcd(e, z) = 1
    for e in [3, 5, 17, 65537]:
        if 1 < e < z and gcd(e, z) == 1:
            break
    else:
        # If none found, choose a random one
        possible_es = [i for i in range(2, z) if gcd(i, z) == 1]
        if not possible_es:
            raise ValueError("No valid e found")
        e = random.choice(possible_es)

    # Compute the modular inverse of e
    d = modinv(e, z)
    if d is None:
        raise ValueError("Modular inverse doesn't exist")

    return n, z, e, d
```

- **rsa_encrypt(message, e, n):**
  - Encrypts ASCII message character-by-character.
  - Checks each character fits in modulus n.
  - Computes $c = m^e \bmod n$ for each character.
  - Output: List of encrypted numbers.

- **rsa_decrypt(ciphertext, d, n):**

  - Computes $m = c^d \bmod n$ for each number.

  - Verifies decrypted values are valid ASCII.

  - Decrypts using the private exponent, checking ASCII validity for message reconstruction.

```python
def rsa_encrypt(message, e, n):
    # Encrypt each character in the message using public key (e, n)
    return [pow(ord(char), e, n) for char in message]

def rsa_decrypt(ciphertext, d, n):
    # Decrypt each number in the ciphertext using private key (d, n)
    return ''.join([chr(pow(c, d, n)) for c in ciphertext])
```

```python
# ---------------------- File IO ----------------------

def save_keys(n, e, d):
    # Save public and private keys to files
    with open("public_key.txt", "w") as f:
        f.write(f"{n} {e}\n")
    with open("private_key.txt", "w") as f:
        f.write(f"{n} {d}\n")

def load_public_key():
    # Load public key from file
    with open("public_key.txt") as f:
        n, e = map(int, f.read().split())
    return n, e

def load_private_key():
    # Load private key from file
    with open("private_key.txt") as f:
        n, d = map(int, f.read().split())
    return n, d

def save_message(ciphertext):
    # Save encrypted message to file
    with open("message.txt", "w") as f:
        f.write(' '.join(map(str, ciphertext)))

def load_message():
    # Load encrypted message from file
    with open("message.txt") as f:
        return list(map(int, f.read().split()))

# ---------------------- Menu System ----------------------
```

```python
# ---------------------- Menu System ----------------------

def menu():
    # Display the main menu and handle user choices
    while True:
        print("\n===== RSA Secure Mail System =====")
        print("1. Generate RSA Keys")
        print("2. Send Encrypted Message")
        print("3. Read & Decrypt Message")
        print("4. Exit")

        choice = input("Choose an option: ")

        if choice == '1':
            # Generate RSA keys from two primes
            p = int(input("Enter prime p: "))
            q = int(input("Enter prime q: "))
            if not is_prime(p) or not is_prime(q) or p == q:
                print("Both numbers must be distinct primes.")
                continue
            try:
                n, z, e, d = generate_keys(p, q)
                save_keys(n, e, d)
                print(f"Keys generated and saved.\nPublic Key (n, e): {n}, {e}\nPrivate Key (n, d): {n}, {d}")
            except Exception as ex:
                print(f"Error: {ex}")
```

```python
        elif choice == '2':
            # Encrypt a message and save it
            try:
                n, e = load_public_key()
                message = input("Enter message to encrypt: ")
                ciphertext = rsa_encrypt(message, e, n)
                save_message(ciphertext)
                print("Message encrypted and saved to message.txt")
            except Exception as ex:
                print(f"Error: {ex}")

        elif choice == '3':
            # Decrypt a saved message using private key
            try:
                n, d = load_private_key()
                ciphertext = load_message()
                message = rsa_decrypt(ciphertext, d, n)
                print("Decrypted Message:", message)
            except Exception as ex:
                print(f"Error: {ex}")

        elif choice == '4':
            # Exit the program
            print("Exiting...")
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    menu()
```

4.3 Input/Output System

We implemented file I/O functions to persist keys and encrypted messages:

- Keys are saved in public_key.txt and private_key.txt.
- **Screenshot:**

private_key.txt

```
247 173
```

public_key.txt

```
247 5
```

- Messages are saved as ciphertext in message.txt.

- **Screenshot:**

message.txt

```
154 43 166 166 232 219 223 145 130 79 20 223 79 20 223 184 223 20 43 112 95 43 51 223 77 43 20
20 184 12 43 50
```

- These files facilitate secure message exchange between users or across sessions.

4.4 Input/Output System Screenshot

```
Final Project — python3 RSA.py — 80×24
Last login: Tue Apr 22 13:11:12 on console
(base) tanzilatabassum@MacBookPro ~ % cd /Users/tanzilatabassum/Desktop/Cybersec
urity,MS/Spring 2025/Network Security/Final Project
cd: too many arguments
(base) tanzilatabassum@MacBookPro ~ % cd "/Users/tanzilatabassum/Desktop/Cyberse
curity,MS/Spring 2025/Network Security/Final Project"

(base) tanzilatabassum@MacBookPro Final Project % python3 RSA.py


===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option:
```

```
Final Project — python3 RSA.py — 80×24
(base) tanzilatabassum@MacBookPro ~ % cd "/Users/tanzilatabassum/Desktop/Cyberse
curity,MS/Spring 2025/Network Security/Final Project"

(base) tanzilatabassum@MacBookPro Final Project % python3 RSA.py


===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 1
Enter prime p: 13
Enter prime q: 19
Keys generated and saved.
Public Key (n, e): 247, 5
Private Key (n, d): 247, 173

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option:
```

Final Project — python3 RSA.py — 80×24

```
curity,MS/Spring 2025/Network Security/Final Project"

(base) tanzilatabassum@MacBookPro Final Project % python3 RSA.py


===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 1
Enter prime p: 13
Enter prime q: 19
Keys generated and saved.
Public Key (n, e): 247, 5
Private Key (n, d): 247, 173

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 2
Enter message to encrypt: Hello! This is a secret Message.
```

Final Project — python3 RSA.py — 80×24

```
3. Read & Decrypt Message
4. Exit
Choose an option: 1
Enter prime p: 13
Enter prime q: 19
Keys generated and saved.
Public Key (n, e): 247, 5
Private Key (n, d): 247, 173

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 2
Enter message to encrypt: Hello! This is a secret Message.
Message encrypted and saved to message.txt

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option:
```

```
Final Project — python3 RSA.py — 80×24

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 2
Enter message to encrypt: Hello! This is a secret Message.
Message encrypted and saved to message.txt

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 3
Decrypted Message: Hello! This is a secret Message.

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option:
```

```
Final Project — -zsh — 80×24
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 2
Enter message to encrypt: Hello! This is a secret Message.
Message encrypted and saved to message.txt

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 3
Decrypted Message: Hello! This is a secret Message.

===== RSA Secure Mail System =====
1. Generate RSA Keys
2. Send Encrypted Message
3. Read & Decrypt Message
4. Exit
Choose an option: 4
Exiting...
(base) tanzilatabassum@MacBookPro Final Project %
```

4.5 Important Technical Details

1. Modular Exponentiation: Uses Python's 3-argument `pow(c, d, n)` for efficient (c^d mod n) calculation.
2. ASCII Handling:
   - Minimum n=128 ensures all ASCII values (0-127) can be encrypted.

    - Explicit checks prevent invalid characters/values.
3. Error Handling:
    - Covers: invalid primes, small modulus, character overflow.
    - Provides descriptive error messages.
4. Security Considerations:
    - Uses cryptographically strong primes.
    - Properly implements RSA mathematical foundations.
    - Prevents common edge cases that could break encryption.

This implementation maintains the security properties of RSA while being practical for educational use with ASCII text. Each function serves a specific, well-defined purpose in the RSA process.

5. Professor Feedback & Project Adaptation

In consultation with our professor, we received feedback encouraging us to think beyond the core RSA implementation. In particular, the professor highlighted the growing relevance of quantum computing and suggested considering cryptographic approaches that could resist quantum-based attacks. Additionally, practical extensions like applying our system to real-world communication tools (e.g., encrypted email) were proposed.

We responded by researching the quantum threat landscape and incorporating post-quantum cryptography (PQC) into our report and future work plan. This significantly broadened the scope of our project and allowed us to bridge theoretical cryptography with practical security challenges.

6. Quantum Computing Threat to RSA

6.1 Vulnerability of RSA

RSA's security is based on the assumption that factoring large integers is computationally infeasible using classical computers. However, quantum algorithms—particularly Shor's Algorithm—can factor these integers in polynomial time, rendering RSA and similar schemes (e.g., ECC) effectively insecure.

Even though quantum computers capable of breaking RSA do not yet exist at scale, adversaries could store encrypted data today and decrypt it in the future—an attack known as "harvest now, decrypt later."

6.2 Need for Post-Quantum Cryptography

To prepare for the post-quantum era, the cryptographic community is actively developing quantum-resistant algorithms that do not rely on integer factorization or discrete logarithms. These include:

- Hash-based cryptography
- Lattice-based cryptography
- Code-based cryptography
- Multivariate polynomial cryptography

7. Post-Quantum Cryptographic Techniques

7.1 Hash-Based Cryptography

Example: SPHINCS+, XMSS
Strength: Digital signatures built on hash functions, which remain secure under quantum attacks.
Limitation: Not used for encryption; large signature sizes.
Implementation Highlight: We demonstrated a Lamport signature scheme in Python as a conceptual example of hash-based signing.

Example Implementation:

```python
# Simplified Lamport Signature Scheme (Conceptual)
import hashlib

class HashBasedSignature:
    def __init__(self):
        self.private_key = [os.urandom(32) for _ in range(256*2)]  # 512 keys for 256-bit hash
        self.public_key = [hashlib.sha256(k).digest() for k in self.private_key]

    def sign(self, message):
        hash_msg = hashlib.sha256(message).digest()
        bits = ''.join(format(byte, '08b') for byte in hash_msg)
        return [self.private_key[i] for i, bit in enumerate(bits)]

    def verify(self, message, signature):
        hash_msg = hashlib.sha256(message).digest()
        bits = ''.join(format(byte, '08b') for byte in hash_msg)
```

```
        return all(hashlib.sha256(sig).digest() == self.public_key[i]
                for i, (sig, bit) in enumerate(zip(signature, bits))
```

7.2 Lattice-Based Cryptography

Example: Kyber, NTRU
Strength: Secure against quantum attacks with practical performance.
Highlight: Kyber512 was selected by NIST as a standard for post-quantum key encapsulation.
Demonstration: We used `liboqs` Python bindings to generate key pairs and encrypt shared secrets.

Example Implementation:

```python
# Kyber Encryption using liboqs (Python Bindings)
from oqs import KeyEncapsulation

# Key Generation
kem = KeyEncapsulation('Kyber512')
public_key = kem.generate_keypair()

# Encryption
ciphertext, shared_secret = kem.encap_secret(public_key)

# Decryption
shared_secret_dec = kem.decap_secret(ciphertext)
```

8. Integration with RSA System

We designed a hybrid cryptographic approach to integrate post-quantum techniques without abandoning the RSA structure.

8.1 Hybrid Encryption Strategy

```
def hybrid_encrypt(message, rsa_pub, pq_pub):
    rsa_cipher = rsa_encrypt(message, rsa_pub['e'],
rsa_pub['n'])
    pq_cipher = pq_encrypt(message, pq_pub)
    return (rsa_cipher, pq_cipher)
```

This dual-encryption method ensures backward compatibility while introducing quantum resistance.

Example Implementation:

```python
def hybrid_encrypt(message, rsa_pub, pq_pub):
    # Encrypt with RSA and post-quantum
    rsa_cipher = rsa_encrypt(message, rsa_pub['e'], rsa_pub['n'])
    pq_cipher = pq_encrypt(message, pq_pub)
    return (rsa_cipher, pq_cipher)

def hybrid_decrypt(ciphertext, rsa_priv, pq_priv):
    # Prioritize post-quantum if RSA is compromised
    try:
        return rsa_decrypt(ciphertext[0], rsa_priv['d'], rsa_priv['n'])
    except QuantumAttackDetectedError:
        return pq_decrypt(ciphertext[1], pq_priv)
```

8.2 Transition Strategy

- **Short-Term:** Introduce hash-based signatures for authenticity.
- **Mid-Term:** Use hybrid encryption (RSA + Kyber).
- **Long-Term:** Fully transition to lattice-based encryption and modular cryptographic architecture.

Full Transition to Post-Quantum:

Replace RSA entirely with lattice-based encryption:

```python
def generate_pq_keys():
    kem = KeyEncapsulation('Kyber512')
    public_key = kem.generate_keypair()
    return public_key, kem

def pq_encrypt(message, public_key):
    kem = KeyEncapsulation('Kyber512')
    ciphertext, shared_secret = kem.encap_secret(public_key)
    return (ciphertext, aes_encrypt(message, shared_secret))  # Use AES for bulk
encryption

def pq_decrypt(ciphertext, kem):
    shared_secret = kem.decap_secret(ciphertext[0])
    return aes_decrypt(ciphertext[1], shared_secret)
```

```

## 9. Challenges in Quantum Migration

| Challenge | Description |
|---|---|
| Key Sizes | Kyber512 keys (~800 bytes) vs RSA-2048 (~256 bytes); needs I/O and storage changes. |
| Performance | PQC algorithms may require more time and memory. |
| Compatibility | Existing systems and protocols may not support PQC out of the box. |
| Algorithm Agility | Systems must support multiple encryption schemes for smooth transition. |
| We proposed an `ALGORITHM_REGISTRY` to dynamically switch between RSA and PQ algorithms as needed. | |

## 10. Future Work and Applications

- Real-World Use: Integrate the system into email clients or messaging platforms to demonstrate practical encryption.
- SPHINCS+ Integration: Add full digital signing and verification.
- Web Interface: Develop a GUI or web-based frontend for broader accessibility.
- Modular Architecture: Allow dynamic switching between RSA and PQC algorithms for educational and security use.
```

11. Conclusion

This project began as a straightforward implementation of RSA but evolved into a deep exploration of secure communication in the quantum age. Through coding, analysis, and professor guidance, we built a secure mail system, identified future threats, and proposed solutions using cutting-edge cryptography. The integration of post-quantum concepts positions this project as not just a technical demonstration but a forward-looking study in cybersecurity.

By adopting a hybrid strategy and anticipating cryptographic advancements, we believe our system sets the groundwork for secure communication in both current and future threat models.

12. References

- NIST Post-Quantum Cryptography Standardization:
  https://csrc.nist.gov/projects/post-quantum-cryptography
- OpenQuantumSafe (OQS) Library:
  https://github.com/open-quantum-safe/liboqs
  https://open-quantum-safe.org/liboqs-python/
- Hash-Based Cryptography:
  https://sphincs.org/
  https://datatracker.ietf.org/doc/html/rfc8391
  https://datatracker.ietf.org/doc/html/rfc8554
- Lattice-Based Cryptography:
  https://pq-crystals.org/kyber/
  https://ntru.org/
- Quantum Computing Threats:
  Shor's Algorithm: Original Paper: Shor, P. W. (1997). Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. arXiv:quant-ph/9508027
- NIST Report on Quantum Threats:
  NISTIR 8105: Report on Post-Quantum Cryptography
- Hybrid Cryptography:
  IETF Draft on Hybrid Key Exchange: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/
- NIST Guidance on Hybrid Systems:
  https://csrc.nist.gov/publications/detail/white-paper/2020/12/15/getting-ready-for-post-quantum-cryptography/final
- Quantum Key Distribution (QKD):
  https://www.etsi.org/images/files/ETSIWhitePapers/QuantumSafeWhitepaper.pdf
- Practical Implementation Guides:
  https://www.cisa.gov/post-quantum-cryptography